GP-1 Report

Group 16

Members :

Simen Haug (shau081)

Bernardino Soesanto (bsoe838)

Introduction

The ReCOP (Reconfigurable Compact Processor) project aims to implement a custom-designed processor on the DE1-SoC FPGA. This project focuses on the development of the processor, including the datapath, control unit, memory interface, and external I/O integration. The processor follows a multicycle architecture and supports a custom instruction set with various addressing modes. The implementation also involves developing an assembler to convert human-readable assembly code into machine-level instructions stored in memory via .mif file. Throughout the project, the ReCOP processor is tested both in simulation using Questa and on hardware via Quartus to verify its functionality. External interfaces such as LEDs and switches are included for real-time I/O operations. This report summarizes the customization processor's architecture, instruction flow, memory interactions, and simulation results.

Additions and Customizations Compared with Original ReCOP Processor

Memory Customization

- Program Memory: The original ReCOP uses a 16-bit wide program memory. In this implementation, it is increased to 32 bits, where each instruction is stored in a single 32-bit word, simplifying memory access and instruction fetch logic.

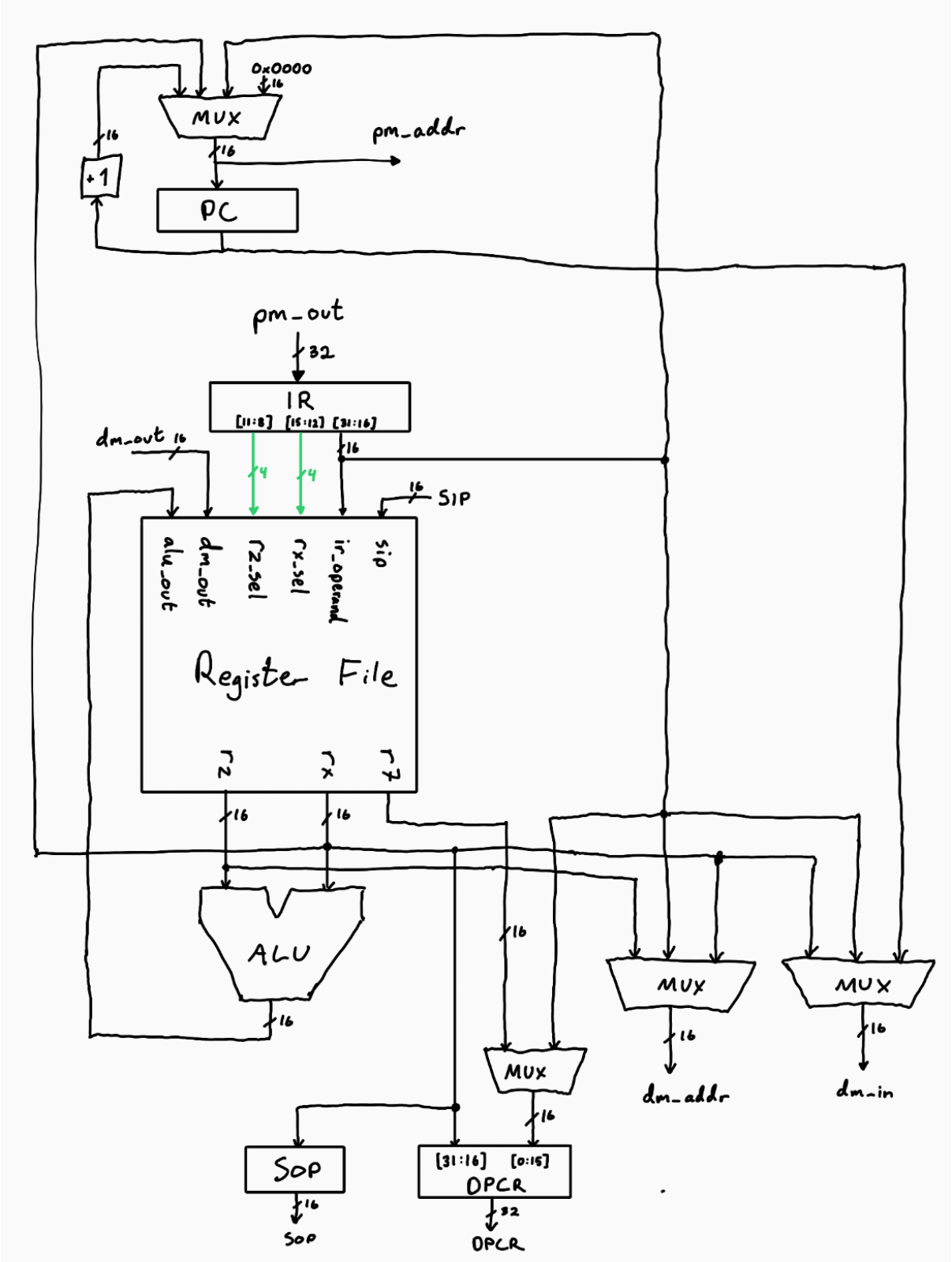
Excluded Features from Original Design

The following components present in the original ReCOP model are not included in this implementation:

- EOT, DPC, and ER flags
- DPRR2
- CLR_IRQ
- SVOPi
- MAX unit for maximum value calculation

These modules were excluded to reduce design complexity and resource usage on FPGA, and because they were not required for the scope of GP1.

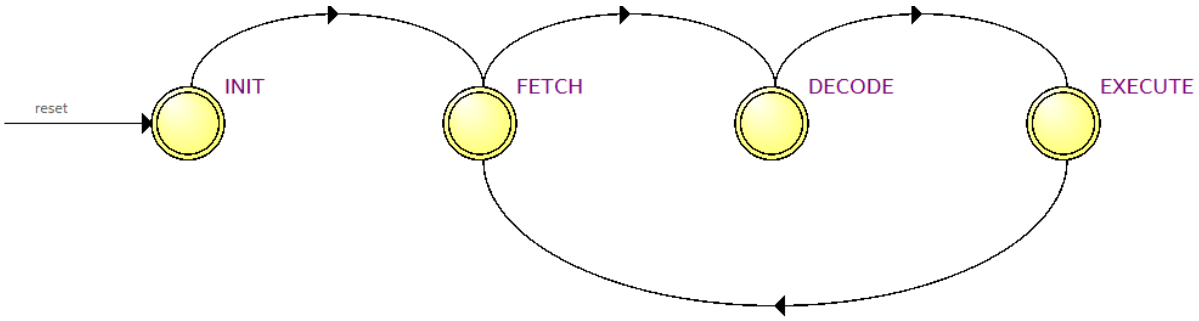
Datapath



The datapath is shown in the diagram above. One thing to note is that the input of the

program counter is fed to the program memory address. The reason for this is that the ROM IP includes a mandatory address input register. We cannot read directly from this register though, so the PC register in the datapath mirrors the address register of the data memory. The Register file includes multiplexers to select the operands Rx and Rz based on control signals. They can either be the registers R0-R15, data memory output, instruction register operand or the SIP.

Control unit

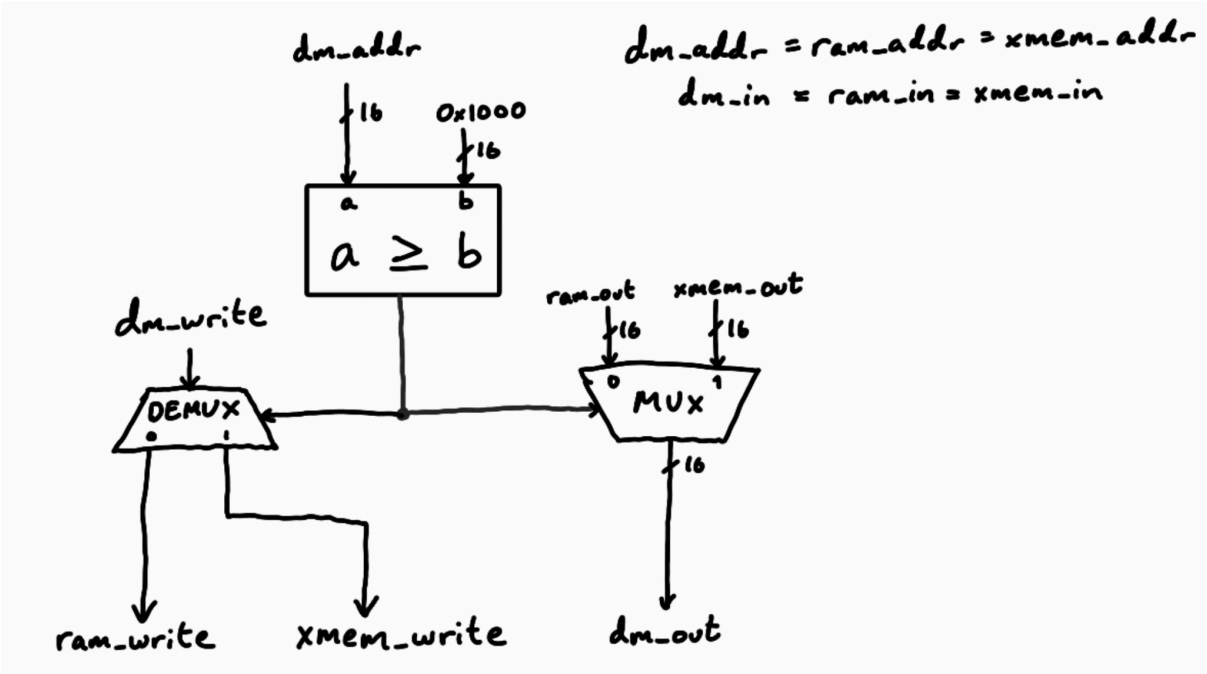


The control unit state machine is shown above.

- INIT: during this state, the registers are set to their reset value and the PC is loaded with the start-of-program address.
- FETCH: This state fetches the instruction to the IR. It also increments the PC to prepare for the next instruction.
- DECODE: During this state the data memory address is set up to prepare for reading from RAM in the EXECUTE state. This state is only needed because the input of the RAM is registered. This means that when doing a LDR (read from RAM) instruction, the output will not be ready before the next clock cycle when in the EXECUTE state.
- EXECUTE: this state performs the instruction itself.

External Interface:

In addition to the standard ReCOP external interface (SOP, SIP, DPCR) this implementation also includes an external memory interface (xmem) as shown in the figure below.



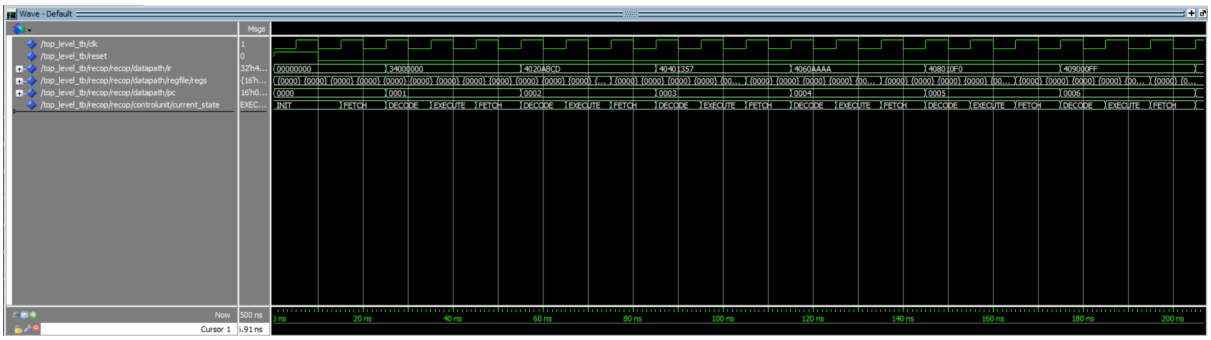
Our RAM has a depth of 4096, so the address range is 0x0000-0x0FFF, when using an address above this range, the external memory interface will be used. This means that the write signal will be sent to either ram_write or xmem_write and the data memory output that the ReCOP sees will either be the RAM output or the external memory interface output.

Assembler

A custom assembler was developed to generate .mif files compatible with the 32-bit program memory. The assembler was written in python and can be found in the assembler/ directory along with example assembly programs.

Testing

- All instructions are verified to work correctly in simulation (Questa) using a memory initialization file containing all instructions. The generated waveforms were analyzed to make sure the instructions behaved as expected



Integration and Deployment

- The procedure to upload the ReCOP files to the FPGA and run an assembly program is described in the README.md file.
- Board testing was performed using a DE1-SoC FPGA board with LED and switch I/O mapped via memory-mapped addresses (e.g., 0x4100 for LED, 0x4200 for switch). In the assembly/ directory there is a file called counter.asm. This displays a running counter using the HEX displays on the FPGA board. In addition it turns on the LEDs based on the switches. The image below shows said program running on the FPGA board.